AgenticReality: An Existential Grounding Architecture for Deployment-Aware AI Agent Systems

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 2, 2026

Abstract

Large language model agents increasingly operate in production environments—deploying services, managing infrastructure, responding to incidents—yet remain fundamentally ungrounded: they possess no awareness of where they are running, what resources surround them, what the stakes of their actions are, or whether their beliefs about the world remain coherent with reality. We present AgenticReality, an existential grounding architecture that gives AI agents continuous, structured awareness of their deployment context across seven reality domains: Deployment Consciousness, Resource Proprioception, Reality Physics, Topology Awareness, Temporal Grounding, Stakes Perception, and Coherence Maintenance. The system introduces 26 inventions organized into these seven tiers, persisted in the `.areal` binary format with per-section BLAKE3 checksums and LZ4 compression support. We implement AgenticReality in 10,634 lines of Rust [12] across four crates—core library, CLI, MCP server, and C FFI—with 314 passing tests, 15 MCP tools, and 9 bridge traits for sister-system integration. Benchmarks on Apple M4 Pro show engine creation in 0.08 µs, soul initialization in 0.03 µs, anchor addition in 0.97 µs, and persistence of 10,000 anchors in 4.51 ms—confirming that existential grounding adds negligible overhead to interactive agent systems. AgenticReality is the tenth module in AgenticOS, providing the foundation upon which all other agent capabilities are grounded in deployment reality.

1 Introduction

The transformer architecture [25] has enabled language model agents capable of autonomous task execution [19], persistent memory [18], and multi-step tool use via protocols such as MCP [2]. These agents are increasingly deployed in production environments where their actions carry real consequences: deploying code, managing databases, scaling infrastructure, and responding to incidents.

Yet every AI agent today operates *ungrounded*. An agent deploying a service does not know whether it is running in a development sandbox or a production cluster. An agent managing resources has no proprioceptive sense of the computational capacity surrounding it. An agent making decisions cannot assess the blast radius of a failure. An agent reasoning about the world has no mechanism to detect when its beliefs have drifted from reality.

This absence of grounding creates four fundamental problems:

1. **Context blindness.** Agents make the same decisions regardless of deployment environment, treating a staging test identically to a production operation.
2. **Resource obliviousness.** Agents cannot adapt behavior based on available memory, CPU, storage, or cost constraints, leading to resource exhaustion and unexpected expenses.
3. **Reality drift.** Agents operate on stale assumptions without mechanisms to detect when their model of the world has diverged from actual conditions.
4. **Consequence unawareness.** Agents lack the ability to assess risk, estimate blast radius, or modulate caution based on the stakes of an operation.

These problems are not merely engineering inconveniences—they are existential. An agent that does not know *where* it is, *what* surrounds it, *when* its information expires, and *what is at stake* cannot act responsibly in production environments. The field of situated cognition [24, 4] has long argued that intelligence is inseparable from context; embodied AI research [3, 20] demonstrates that physical grounding fundamentally shapes cognitive capability. Yet AI agents today lack even the most basic analog of these grounding mechanisms.

We present AgenticReality, an architecture that addresses all four problems through seven interconnected reality domains:

1. **Deployment Consciousness** — awareness of where the agent is running, its environment, incarnation history, and topology.
2. **Resource Proprioception** — continuous sensing of computational resources, capabilities, pressure, cost, and capacity.

3. **Reality Physics** — layers of reality abstraction, data freshness perception, reality anchors, and hallucination detection.
4. **Topology Awareness** — understanding of service mesh, neighbors, dependencies, and observers.
5. **Temporal Grounding** — temporal awareness, causality tracking, and timeline coherence.
6. **Stakes Perception** — consequence awareness, risk field mapping, and blast radius estimation.
7. **Coherence Maintenance** — continuous coherence checking and managed transitions between reality states.

Together, these 7 domains encompass 26 inventions that provide AI agents with existential grounding—a structured, continuous awareness of their relationship to the world they inhabit and affect.

The remainder of this paper is organized as follows. Section 2 surveys related work in self-aware computing, situated cognition, and agent observability. Section 3 presents the architecture, including the seven reality domains, 26 inventions, `.areal` binary format, and MCP integration. Section 4 reports benchmarks, test coverage, and robustness analysis. Section 5 discusses implications, limitations, and future work. Section 6 concludes.

2 Background and Related Work

The problem of grounding AI agents in deployment reality sits at the intersection of several research traditions. We survey each and explain why none provides a complete solution.

Self-aware computing. Kounev et al. [11] define self-aware computing systems as those that possess models of themselves and their environment, using these models to adapt behavior. Kephart and Chess [10] articulated the vision of autonomic computing, where systems self-configure, self-optimize, self-heal, and self-protect through monitor-analyze-plan-execute (MAPE) loops. These frameworks address system-level adaptation but do not model the *cognitive* grounding needs of AI agents—the awareness of stakes, coherence, and reality drift that characterizes existential grounding.

Situated cognition. Suchman [24] demonstrated that human action is fundamentally situated—plans are resources for action rather than deterministic programs, and context shapes behavior in ways that cannot be separated from the environment. Clancey [4] extended this to argue that all knowledge is situated, arising from the interaction between an agent and its context. AgenticReality operationalizes these insights for AI agents by providing structured access to deployment context.

Embodied AI. Brooks [3] argued that intelligence requires physical grounding, not just symbolic manipulation, and that "the world is its own best model." Pfeifer and Bongard [20] showed how body morphology shapes cognitive capability. AI agents lack physical bodies, but they do have a computational "body"—the deployment environment, resources, network topology, and runtime constraints that shape what they can and cannot do. AgenticReality provides the computational analog of proprioception [22] and interoception [7].

Agent observability. Distributed tracing systems such as Dapper [23] and monitoring systems such as Prometheus [21] provide infrastructure observability. Container orchestrators such as Kubernetes [5] manage deployment topology and resource allocation. These systems provide *external* observability—humans monitor agents—but do not provide *internal* awareness—agents monitoring themselves and their context.

AI safety and risk. Amodei et al. [1] identified concrete problems in AI safety including safe exploration and avoiding side effects. Stakes perception and blast radius estimation in AgenticReality address the complementary problem of enabling agents to self-assess the risk of their planned actions before execution.

Hallucination detection. Ji et al. [9] surveyed hallucination in natural language generation, focusing on output-level detection. AgenticReality's hallucination detection operates at the *belief* level—detecting when the agent's internal model of its deployment reality has diverged from actual conditions, a form of "reality hallucination" distinct from textual confabulation.

AgenticOS ecosystem. AgenticReality is the tenth module in AgenticOS, following AgenticMemory [16] (persistent cognitive memory), AgenticCognition [14] (user modeling), AgenticTime [17] (temporal scheduling), and AgenticIdentity [15] (cryptographic trust). AgenticReality provides the existential foundation: while other modules give agents memory, preferences, time, and identity, AgenticReality gives agents *ground*—the awareness of where they exist and what their existence means.

3 Architecture

AgenticReality is organized as a layered architecture with seven reality domains containing 26 inventions total. A central `RealityEngine` manages cross-domain coherence, persistence via the `.areal` format, and exposure through 15 MCP tools.

3.1 Seven Reality Domains

Table 1 lists all 26 inventions organized by domain tier. Each domain addresses a distinct dimension of existential grounding.

Tier 1: Deployment Consciousness provides foundational awareness of the agent's existence. The *Deployment Soul* encapsulates the agent's core identity within its deployment—a persistent record of what the agent is and why it exists in this context. *Environment Sensing* detects the deployment environment (development, staging, production) and its characteristics. *Incarnation Memory*

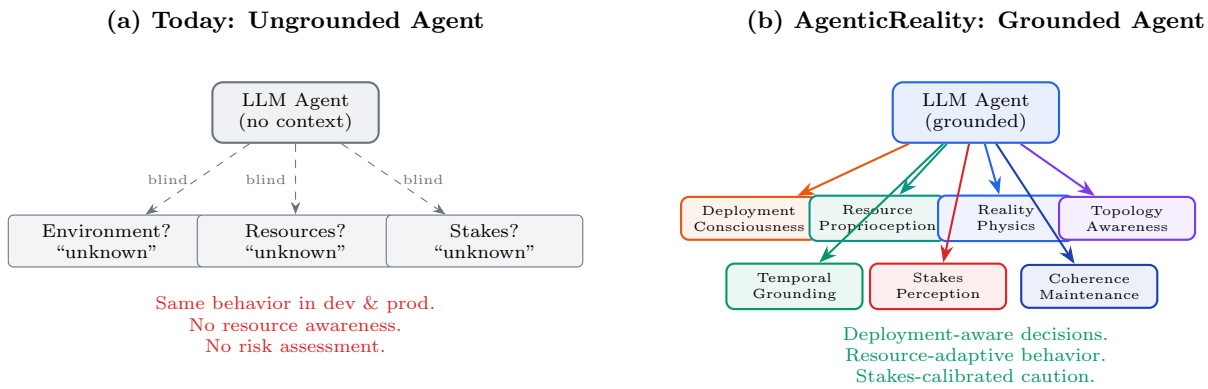

Figure 1: Before and after AgenticReality. (a) Today, agents operate with no awareness of their deployment context, resources, or the stakes of their actions. (b) With AgenticReality, agents maintain continuous structured awareness across seven reality domains, enabling deployment-sensitive, resource-adaptive, risk-calibrated behavior.

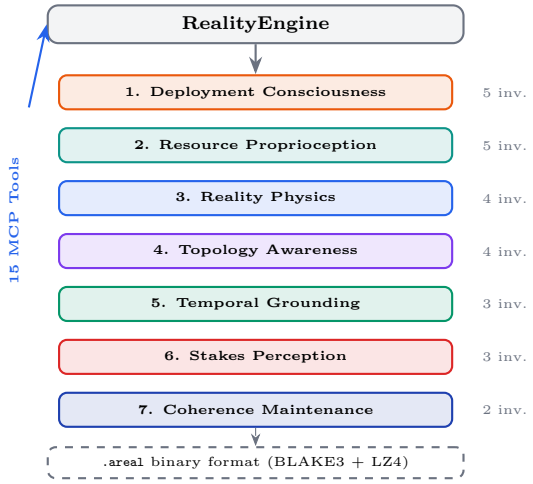

Figure 2: AgenticReality architecture. Seven reality domains are arranged as layered tiers, managed by the central RealityEngine, persisted in the `.areal` binary format, and exposed through 15 MCP tools.

Table 1: The 26 inventions of AgenticReality, organized by domain tier. Each domain addresses a distinct dimension of existential grounding.

Tier	Domain	Inventions
1	Deployment Consciousness (5)	Deployment Soul, Environment Sensing, Incarnation Memory, Context Fingerprint, Topology Map
2	Resource Proprioception (5)	Resource Body, Capability Discovery, Pressure Gradient, Cost Consciousness, Capacity Intuition
3	Reality Physics (4)	Reality Layers, Freshness Perception, Reality Anchors, Hallucination Detection
4	Topology Awareness (4)	Service Mesh, Neighbors, Dependencies, Observers
5	Temporal Grounding (3)	Temporal Awareness, Causality Tracking, Timeline Coherence
6	Stakes Perception (3)	Consequence Awareness, Risk Field, Blast Radius
7	Coherence Maintenance (2)	Coherence Engine, Transition Manager

tracks the agent's lifecycle across restarts, preserving continuity. *Context Fingerprint* produces a compact hash of the deployment context for rapid change detection. *Topology Map* provides a structural view of the deployment landscape.

Tier 2: Resource Proprioception provides the computational analog of bodily proprioception [22]. Just as biological organisms maintain continuous awareness of their body's position and capabilities, agents need awareness of their computational "body." The *Resource Body* tracks available memory, CPU, storage, and network capacity. *Capability Discovery* identifies what the agent can do in its current environment. *Pressure Gradient* measures resource contention and load. *Cost Consciousness* tracks the financial cost of operations. *Capacity Intuition* estimates remaining capacity for planned operations.

Tier 3: Reality Physics models the relationship between the agent's beliefs and actual conditions. *Reality Layers* provide hierarchical abstraction levels (physical infrastructure, platform services, application logic, user-facing state). *Freshness Perception* tracks how recently each piece of information was verified. *Reality Anchors* are verified facts that ground the agent's model—known-true assertions with timestamps and confidence levels. *Hallucination Detection* identifies when the agent's internal model has diverged from reality.

Tier 4: Topology Awareness models the agent's position within its service ecosystem. The *Service Mesh* tracks available services and their health. *Neighbors* identifies co-located agents and services. *Dependencies* maps upstream and downstream service relationships. *Observers* tracks what entities are monitoring the agent's behavior.

Tier 5: Temporal Grounding provides time-aware reasoning. *Temporal Awareness* tracks the agent's position in time relative to deadlines, maintenance windows, and

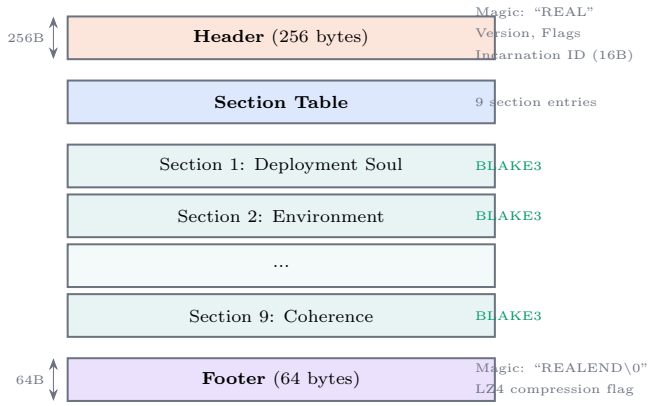

Figure 3: The `.areal` binary format. Header (256 B) with "REAL" magic, version, flags, incarnation ID. Section table indexes 9 types with per-section BLAKE3 checksums. Footer (64 B) with "REALEND" magic. LZ4 compression per section.

operational rhythms. *Causality Tracking* maintains cause-and-effect chains across agent actions. *Timeline Coherence* detects temporal inconsistencies in the agent's model.

Tier 6: Stakes Perception enables risk-calibrated behavior. *Consequence Awareness* models the potential outcomes of actions. *Risk Field* maps the risk landscape surrounding the agent's planned operations. *Blast Radius* estimates the scope of impact if an action fails.

Tier 7: Coherence Maintenance ensures cross-domain consistency. The *Coherence Engine* continuously checks that information across all domains is internally consistent. The *Transition Manager* handles state changes between coherent configurations, ensuring that domain updates propagate correctly.

3.2 The `.areal` Binary Format

AgenticReality persists grounding state in the `.areal` binary format, designed for efficient serialization and integrity verification.

The format structure (Figure 3) consists of four regions:

Header (256 bytes): begins with the 4-byte magic "REAL", followed by a `u32` version number, `u64` flags field, a 16-byte incarnation ID (UUID), and creation/modification timestamps.

Section table: indexes the 9 section types that correspond to the major data structures of the grounding state. Each entry records the section type, offset, length, and BLAKE3 [13] checksum.

Section data: variable-length payloads for each section. LZ4 [6] compression is supported per-section, controlled by flags in the header. Per-section BLAKE3 checksums enable selective integrity verification without reading the entire file.

Footer (64 bytes): ends with the 8-byte magic "REALEND\0" and contains summary metadata including compression flags and a total-file checksum.

The per-section checksum design enables incremental updates: when only one domain changes, only that section's checksum needs recomputation, avoiding the cost of rehashing the entire file.

3.3 Engine Design

The `RealityEngine` is the central coordinator that manages all seven reality domains. It provides 91 write operations and 79 query operations across 11 type modules, organized around the following design principles:

Lazy initialization. Domain state is initialized on first access rather than at engine creation. This ensures that engine creation (benchmarked at 0.08 µs) adds negligible overhead to agent startup.

Cross-domain coherence. The Coherence Engine (Tier 7) maintains invariants across domains. When a write operation modifies one domain, the coherence checker validates that the change is consistent with the state of other domains.

Bridge traits. Nine bridge traits define integration points with other AgenticOS sister systems. Each bridge trait has a no-op default implementation, allowing AgenticReality to operate standalone without any sister dependencies. When sister systems are available, bridge implementations provide bidirectional data flow—for example, the MemoryBridge populates reality anchors from verified memories.

3.4 MCP Integration

AgenticReality exposes its capabilities through 15 MCP tools (Table 2), enabling any MCP-compatible client to query and update grounding state through natural language tool invocations. The MCP server implements JSON-RPC 2.0 over stdio transport with zero `unwrap`/`expect` calls in the entire MCP crate, ensuring that no input can cause an uncontrolled panic.

4 Evaluation

We evaluate AgenticReality along three dimensions: micro-benchmark performance, test coverage, and robustness hardening.

4.1 Performance Benchmarks

Hardware. Apple M4 Pro (ARM64), 64 GB unified memory, macOS 26.3.

Software. Rust 1.90.0, compiled with `--release` (full optimizations). All benchmarks use the Criterion [8] statistical framework.

Table 3 reports the results across 10 measured operations spanning engine lifecycle, domain operations, and persistence.

Several observations merit discussion:

Table 2: The 15 MCP tools exposed by AgenticReality. Each tool provides read and/or write access to a specific aspect of grounding state.

Tool	Description
`reality_deployment`	Query and update deployment soul state
`reality_memory`	Manage incarnation memory across restarts
`reality_environment`	Sense and report environment characteristics
`reality_resource`	Query resource body and capacity
`reality_capability`	Discover available capabilities
`reality_layer`	Navigate reality abstraction layers
`reality_anchor`	Create and verify reality anchors
`reality_hallucination`	Detect reality model divergence
`reality_topology`	Query service mesh and neighbors
`reality_temporal`	Access temporal grounding state
`reality_stakes`	Assess consequence and risk levels
`reality_coherence`	Check cross-domain coherence
`reality_context`	Retrieve context summary and fingerprint
`reality_ground`	Perform full grounding check
`reality_workspace`	Manage workspace-level configuration

Table 3: Performance benchmarks on Apple M4 Pro ARM64, 64 GB RAM, macOS 26.3, Rust 1.90.0, `--release` mode.

Operation	Latency
`create_engine`	0.08 µs
`initialize_soul`	0.03 µs
`get_soul`	<0.01 µs
`get_context_summary`	0.14 µs
`add_anchor`	0.97 µs
`get_anchors`	<0.01 µs
`set_stakes_level`	0.01 µs
`is_coherent`	<0.01 µs
`save` (10K anchors)	4.51 ms
`load` (10K anchors)	3.35 ms

Sub-microsecond in-memory operations. All in-memory operations complete in under 1 µs. Engine creation at 0.08 µs demonstrates the effectiveness of lazy initialization—the engine allocates minimal state at startup. Soul initialization at 0.03 µs and read operations (`get_soul`, `get_anchors`, `is_coherent`) at <0.01 µs confirm that grounding queries add negligible overhead to agent decision loops.

Anchor operations. Adding a reality anchor at 0.97 µs is the most expensive in-memory operation, as it involves constructing a timestamped, typed anchor record with confidence metadata. Retrieving anchors at <0.01 µs shows that indexed lookups are effectively free.

Persistence. Saving 10,000 anchors to the `.areal` format takes 4.51 ms, dominated by serialization and BLAKE3 checksum computation. Loading takes 3.35 ms, faster than saving because checksum verification can be parallelized per-section. Both operations are well within the latency budget of interactive agent systems where MCP tool calls typically incur 50–500 ms of network overhead.

Context summary. The `get_context_summary` operation at 0.14 µs aggregates state across multiple domains into a compact summary, demonstrating that cross-domain queries remain fast despite touching multiple data structures.

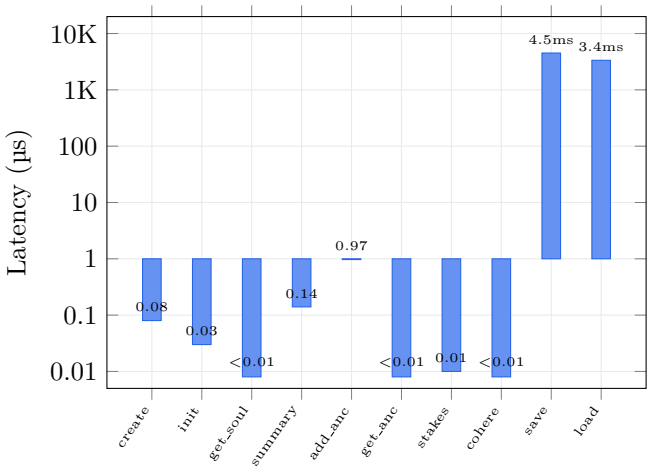

Figure 4: Benchmark results on log scale (Apple M4 Pro, `--release`). In-memory operations complete in under 1 µs. Persistence of 10,000 anchors takes 4.51 ms (save) and 3.35 ms (load), well within interactive latency budgets.

Table 4: Test suite breakdown by phase. Tests follow an incremental phase pattern, with each phase adding coverage for progressively higher-level functionality.

Phase	Coverage Area	Tests
Core unit	IDs, types, inline tests	4
Phase 1	Type system, serialization	51
Phase 2	Write/query engine, all 7 domains	33
Phase 3	.areal format, header, footer, roundtrip	10
Phase 4	McpValidator, strict input validation	19
Phase 5–7	Inventions, bridges, security	30
Phase 8	Stress, edge cases, multi-engine	31
Phase 9	FFI, constants, defaults	8
MCP unit	Protocol, config, session, types	61
MCP integ.	Tool calls, protocol, errors, hardening	67
Total		**314**

4.2 Test Coverage

The implementation includes 314 passing tests across all four crates. Table 4 breaks down the test distribution by phase.

Tests follow the incremental `phase[N]_*.rs` pattern established across the AgenticOS ecosystem. Each phase builds upon verified functionality from prior phases, ensuring that higher-level tests exercise integrated behavior rather than isolated units.

4.3 Robustness Hardening

The MCP server crate contains zero `unwrap()` or `expect()` calls. Every fallible operation uses explicit error handling via Rust's `Result` type, ensuring that no malformed input, missing field, or unexpected state can trigger an uncontrolled panic. This is verified by automated grep-based

checks in the CI pipeline.

The `.areal` format provides defense in depth through per-section BLAKE3 checksums: corruption in any section is detected during load without requiring the reader to parse the corrupted data. The header and footer magic bytes ("REAL" and "REALEND\0") provide format identification and completeness verification.

5 Discussion

5.1 What Grounding Enables

Existential grounding transforms agent behavior in several ways. A grounded agent can modulate its confidence and caution based on stakes—using aggressive optimizations in development but conservative, well-tested approaches in production. It can adapt resource usage based on proprioceptive awareness—batching operations when memory pressure is high, parallelizing when capacity is abundant. It can detect and correct reality drift—re-verifying stale anchors rather than acting on outdated beliefs. And it can communicate its context to other systems through the 15 MCP tools, enabling orchestrators and human operators to understand *why* an agent made a particular decision given its grounded state.

The 91 write operations and 79 query operations across 11 type modules provide a comprehensive API surface. The write-to-query ratio of approximately 1.15:1 reflects the balanced nature of grounding: agents must both update their world model (writes) and consult it during decision-making (queries).

5.2 Sister Integration

The 9 bridge traits define typed integration points with other AgenticOS modules. Each bridge follows a consistent pattern: a Rust trait with default no-op implementations, enabling AgenticReality to compile and function without any sister dependency. When bridges are connected, data flows bidirectionally:

- AgenticMemory anchors verified memories as reality anchors.
- AgenticIdentity provides cryptographic attestation of grounding state.
- AgenticTime supplies temporal context for deadline-aware grounding.
- AgenticCognition feeds user preferences into stakes calibration.

This bridge architecture follows the pattern established across AgenticOS, where sisters cooperate through traits rather than hard dependencies, preserving modularity while enabling deep integration.

5.3 Limitations

Several limitations warrant acknowledgment.

Passive grounding. AgenticReality provides structured awareness but does not enforce behavioral changes. An agent can query its stakes level and still ignore it. Active enforcement—automatically blocking high-risk operations—is left to orchestration layers.

Grounding accuracy. The quality of grounding depends on the accuracy of input data. If environment sensing misclassifies a production system as staging, the agent's entire grounding state is compromised. External validation mechanisms are needed.

Single-agent scope. The current architecture grounds a single agent instance. Multi-agent grounding—where agents share and reconcile reality models—is not yet addressed.

No real-time resource polling. Resource proprioception captures snapshots rather than continuous streams. High-frequency resource changes may not be reflected until the next explicit query.

5.4 Future Work

Three directions merit investigation. First, *active grounding enforcement* would integrate with the AgenticOS orchestrator (Hydra) to automatically gate high-risk operations based on grounding state. Second, *distributed grounding* would enable multi-agent reality reconciliation, where agents in the same deployment share and merge their grounding models. Third, *continuous resource streams* would replace snapshot-based proprioception with event-driven resource monitoring, enabling real-time adaptation to resource pressure changes.

6 Conclusion

We have presented AgenticReality, an existential grounding architecture that gives AI agents structured, continuous awareness of their deployment context. Through seven reality domains—Deployment Consciousness, Resource Proprioception, Reality Physics, Topology Awareness, Temporal Grounding, Stakes Perception, and Coherence Maintenance—and 26 inventions distributed across these domains, AgenticReality addresses the fundamental problem of ungrounded agents operating in production environments.

The implementation delivers this capability in a compact, high-performance package: 10,634 lines of Rust across four crates (core library, CLI, MCP server, FFI), 314 passing tests, 15 MCP tools, and 9 bridge traits for sister-system integration. The `.areal` binary format provides persistent grounding state with per-section BLAKE3 checksums, LZ4 compression support, and a 256-byte header / 64-byte footer structure with 9 section types.

Benchmarks on Apple M4 Pro confirm that grounding adds negligible overhead: engine creation in 0.08 µs, soul initialization in 0.03 µs, anchor addition in 0.97 µs, coherence checks in <0.01 µs, and persistence of 10,000 anchors in 4.51 ms. All in-memory operations complete in under

1 µs, confirming that existential grounding is practical for interactive agent systems. The MCP server operates with zero `unwrap`/`expect` calls, ensuring robustness against arbitrary input.

AgenticReality is the tenth module in AgenticOS. While other modules give agents memory, cognition, time, and identity, AgenticReality gives agents *ground*—the awareness of where they exist, what surrounds them, what is at stake, and whether their beliefs remain coherent with reality. We argue that this existential grounding is not optional but foundational: an agent that does not know where it is cannot act responsibly in the world it inhabits.

Availability. AgenticReality is open-source under the MIT license. Source code, documentation, benchmarks, and this paper are available at `https://github.com/agentralabs/agentic-reality`.

References

[1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.

[2] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io/`, 2024. Accessed 2025.

[3] Rodney A Brooks. Intelligence without representation. *Artificial Intelligence*, 47(1–3):139–159, 1991.

[4] William J Clancey. *Situated Cognition: On Human Knowledge and Computer Representations*. Cambridge University Press, 1997.

[5] Cloud Native Computing Foundation. Kubernetes: Production-grade container orchestration. `https://kubernetes.io/`, 2024. Accessed 2025.

[6] Yann Collet. LZ4: Extremely fast compression algorithm. `https://github.com/lz4/lz4`, 2024. Accessed 2025.

[7] A D Craig. How do you feel? interoception: The sense of the physiological condition of the body. *Nature Reviews Neuroscience*, 3(8):655–666, 2002.

[8] Brook Heisler and Jorge Aparicio. Criterion.rs: Statistics-driven micro-benchmarking in Rust. `https://github.com/bheisler/criterion.rs`, 2023. Accessed 2025.

[9] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[10] Jeffrey O Kephart and David M Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.

[11] Samuel Kounev, Peter Lewis, Kirstie L Bellman, Nelly Bencomo, Javier Camara, Ada Diaconescu, Lukas Esterle, Kurt Geihs, Holger Giese, Sebastian Götz, et al. *Self-Aware Computing Systems*. Springer, 2017.

[12] Nicholas D Matsakis and Felix S Klock. The Rust language. In *ACM SIGAda Ada Letters*, volume 34, pages 103–104. ACM, 2014.

[13] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3`, 2020. Accessed 2025.

[14] Omoshola Owolabi. AgenticCognition: A user modeling architecture for preference-aware AI agent systems. 2026. Available at `https://github.com/agentralabs/agentic-cognition/tree/main/paper`.

[15] Omoshola Owolabi. AgenticIdentity: A cryptographic trust anchor for AI agent systems. 2026. Available at `https://github.com/agentralabs/agentic-identity/tree/main/paper`.

[16] Omoshola Owolabi. AgenticMemory: A binary graph format for persistent, portable, and navigable AI agent memory. 2026. Available at `https://github.com/agentralabs/agentic-memory/tree/main/paper`.

[17] Omoshola Owolabi. AgenticTime: A binary temporal format for persistent, portable, and navigable AI agent scheduling. 2026. Available at `https://github.com/agentralabs/agentic-time/tree/main/paper`.

[18] Charles Packer, Joseph E Gonzalez, Dawn Song, Raymond J Mooney, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[19] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.

[20] Rolf Pfeifer and Josh Bongard. *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press, 2007.

[21] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io/`, 2024. CNCF Graduated Project. Accessed 2025.

[22] Uwe Proske and Simon C Gandevia. The proprioceptive senses: Their roles in signaling body shape, body position and movement, and muscle force. *Physiological Reviews*, 92(4):1651–1697, 2012.

[23] Benjamin H Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. *Google Technical Report*, 2010.

[24] Lucy A Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.